Task-wise Implementation Report

Case study: AI-Guided Prioritization of Protein Mutations for Stability and Functional Retention
Author: Ambuj Srivastava
Date: 2026-11-05
Repository: `medvolt_protein_engineering/`

This report is organised by assignment task. For each task it states the objective, the implementation, the exact command that reproduces the output, the artifacts written, and the headline result. It is a companion to `report/final_report.md`, which presents the same work as a scientific narrative rather than a task checklist.

Every number quoted here was read from the artifact named beside it. All artifacts in this report were produced by a single clean run at seed `20261102`.

0. Prerequisites

```
cd medvolt_protein_engineering
python -m venv .venv && source .venv/bin/activate
pip install -r requirements-dev.txt
python -c "import sys, sklearn, numpy; print(sys.version.split()[0], sklearn.__version__, numpy.__version__)"
# expected: 3.11.x 1.9.1 2.4.6
```

Everything, end to end — this covers every implemented task except the EDA notebook:

```
python main.py --config config/config.yaml
```

Measured 126 s on an M-series Mac. Nothing is cached between runs, so a re-run costs the same and produces byte-identical numbers.

There is **zero hardcoding**: every path, hyperparameter, threshold and the random seed resolve from `config/config.yaml`, validated on load against the bounds in `src/common/constants.py`. No tuning value appears in any module.

1. Quick reference — one command per task

#	Task	Command	Primary output
1	Exploratory data analysis	`jupyter lab notebooks/exploratory_analysis.ipynb`	the notebook itself (outputs committed)
2	Preprocessing & feature engineering	`python -m src.data_processing --config config/config.yaml`	`data/processed/`
3	Baseline models & benchmark	`python -m src.train --config config/config.yaml`	`results/metrics/baseline_benchmark.md`

#	Task	Command	Primary output
4	Neural networks	`python -m src.train_torch --config config/config.yaml` `python -m src.train_torch_v2 --config config/config.yaml`	`results/metrics/torch_vs_baselines.md` `results/metrics/torch_v2_vs_baselines.md`
5	Evaluation & comparative analysis	`python -m src.evaluate --config config/config.yaml`	`results/metrics/model_comparison.csv`
6	Ablation study	`python -m src.ablation --config config/config.yaml`	`results/metrics/ablation_summary.md`
7	Interpretation & feature attribution	`python -m src.attribution --config config/config.yaml`	`results/metrics/feature_importances.csv`
8	Candidate prioritization & uncertainty	`python -m src.predict --config config/config.yaml`	`results/mutation_rankings/top_10_candidates.csv`
9	Scientific interpretation	`python -m src.interpret --config config/config.yaml`	`results/mutation_rankings/candidate_interpretatior`
10	Research extension	*documentation only — no command*	`README.md` § Research extension

Stages consume the artifacts of the stages above them, so run them in this order or simply use `python main.py`. Selected stages, always executed in dependency order:

```
python main.py --list                # show the stage names
python main.py --stages train evaluate
python main.py --from evaluate       # this stage and everything after it
```

One stage has no assignment task of its own. `validation` writes the CV fold report and the leakage audit that every later stage depends on:

```
python -m src.validation --config config/config.yaml
```

2. Task-by-task detail

Task 1 — Exploratory data analysis

Objective. Understand the dataset before modelling it, and identify the properties that constrain the modelling choices.

Implementation. `notebooks/exploratory_analysis.ipynb`, 13 sections, run headless and committed with outputs. It covers missing values, duplicate mutations, internal consistency, target and feature distributions, outliers, mutation-type distribution, protein/family structure, correlations, embedding content, and candidate-vs-training distribution shift.

Command.

```
jupyter lab notebooks/exploratory_analysis.ipynb

# or re-execute headless, in place:
python -m jupyter nbconvert --to notebook --execute --inplace \
    notebooks/exploratory_analysis.ipynb
```

Outputs. The executed notebook.

Key findings. Three findings determined the rest of the project:

1. **The data is clean.** 840 labeled rows, 180 candidates, no missing values, no infinities, no duplicates, no out-of-range features. No imputation was needed on the data as given.
2. **Mutations are nested in proteins, nested in families** — 840 mutations across 28 proteins across 7 families, a median of 31 mutations per protein. A random split would therefore place mutations from the same protein on both sides of the fold boundary. This is the leakage argument that drives Task 5.
3. **The 256-d embedding is constant within a protein.** It is a per-protein vector joined onto every mutation of that protein, so it carries no within-protein signal and cannot rank two mutations of the same protein. Tasks 6 and 7 later confirmed this quantitatively.

Task 2 — Preprocessing & feature engineering

Objective. Ingest, validate and merge the three input files; engineer physicochemical delta features; export reproducible artifacts.

Implementation. `src/data_processing.py` . Schema validation on ingest, relational merge on `protein_id` , anomaly inspection, one-hot encoding of secondary structure (`ss_H` / `ss_E` / `ss_C`), and seven engineered features: signed and absolute deltas for hydrophobicity, charge and volume, plus `relative_position` (`position / protein_length`).

Encoding is deliberately split from modelling: categorical encoding happens here, but **imputation, scaling and embedding PCA are steps inside the per-fold pipeline**, never applied to the full dataset. That is what keeps the Task 5 fold boundaries honest.

Command.

```
python -m src.data_processing --config config/config.yaml
```

Outputs.

Artifact	Contents
`data/processed/processed_train.parquet`	840 labeled rows, 22 tabular + 256 embedding features
`data/processed/processed_candidates.parquet`	180 candidate rows, same schema minus targets
`data/processed/feature_manifest.json`	Feature lists, group columns, targets, seed, quality report

Key result. 22 tabular features and 256 embedding features. The quality report records `imputed: {}` , `inf_replaced: 0` , `target_rows_dropped: 0` — nothing had to be repaired. It also records the distribution shift that Task 8 acts on: **8 candidate proteins (58 rows) and 2 candidate families, PF08 and PF09 (58 rows), are absent from the labeled data entirely.**

Task 3 — Baseline models & benchmark

Objective. Establish classical baselines for both targets under a scientifically appropriate validation strategy.

Implementation. `src/train.py`. Three model families — ridge, random forest, gradient boosting — fitted for both targets under all three splitting strategies (`random`, `protein_group`, `family_group`), giving 90 pipelines. Each is re-fitted from scratch inside every training fold. Model selection is by `pooled_spearman` on `family_group`, the strictest strategy.

Grouped splitting is enforced, not assumed: `src/validation.py` asserts zero group overlap across every fold boundary and raises `LeakageError` otherwise. `results/metrics/cv_fold_summary.csv` confirms `leaked_proteins = 0` and `leaked_families = 0` for every grouped fold.

Command.

```
python -m src.validation --config config/config.yaml   # fold report + leakage audit
python -m src.train --config config/config.yaml
```

Outputs. `results/metrics/baseline_benchmark.{csv,md}`, `cv_metrics_summary.csv`, `cv_fold_metrics.csv`, `oof_predictions.parquet`, `model_selection.json`, and the fitted pipelines under `results/models/`.

Key result. Gradient boosting wins both targets under `family_group`:

Target	Model	RMSE	R^2	Pearson	Spearman
`stability_ddG`	gradient_boosting	0.426	0.341	0.598	**0.556**
`functional_score`	gradient_boosting	0.069	0.634	0.796	**0.747**

Stability is the harder target throughout — R^2 0.34 against 0.63.

Task 4 — Neural network models

Objective. Implement a neural approach to the same two targets and benchmark it honestly against the baselines.

Implementation. Two architectures, sharing one trainer:

- **v1, multitask MLP** (`src/torch_model.py`, `src/train_torch.py`) — a shared trunk over all 278 features with two task heads.
- **v2, dual-branch disentangled network** (`src/torch_model_v2.py`, `src/train_torch_v2.py`) — a tabular branch, a separate embedding branch compressed through a bottleneck, residual fusion, then two heads. The separation is a direct response to the Task 1 finding that the embedding is per-protein-constant: it stops 256 constant dimensions from swamping 22 informative ones.

Both inherit `FoldTrainer` (`src/torch_training.py`), which owns the epoch loop, early stopping, target scaling and cross-validation. Each subclass supplies only `fit_fold_inputs` and `build_model`.

Neural training has **two** leakage boundaries, not one: the inner validation split used for early stopping is carved out of the training fold and grouped the same way as the outer split, so no protein informs its own stopping epoch.

Command.

```
python -m src.train_torch --config config/config.yaml      # v1 multitask MLP
python -m src.train_torch_v2 --config config/config.yaml   # v2 dual-branch
```

Outputs. Per model: `*_cv_metrics.csv`, `*_oof_predictions.parquet`, `*_training_history.csv`, `*_fold_summary.csv` (the early-stopping epoch per fold), `*_vs_baselines.{csv,md}`, fold weights under `results/models/torch{,_v2}/`, and learning-curve figures.

Key result (`family_group`, pooled Spearman).

Model	`stability_ddG`	`functional_score`
gradient_boosting *(Task 3)*	**0.556**	**0.747**
multitask_nn	0.551	0.218
dual_branch_nn	0.390	0.282

Reported as measured: neither network beats gradient boosting. On stability the multitask MLP is competitive (0.551 vs 0.556); on function both networks fail badly, with $R^2 \leq 0$ — worse than predicting the training mean. 840 rows across 7 families is too little data for these architectures, and the honest conclusion is that gradient boosting is the model to ship. The networks are retained because the comparison is itself a result.

An additional finding: the dual-branch architecture change and its loss-weight change were initially confounded. A controlled 2×2 showed they interact, which is recorded in the final report rather than glossed over.

Task 5 — Model evaluation & comparative analysis

Objective. Compare models on comprehensive metrics along two axes — model family and validation strategy — and quantify what family-aware splitting costs.

Implementation. `src/evaluate.py`. Pooled out-of-fold metrics (RMSE, MAE, R^2, Pearson, Spearman) with per-fold standard deviations, for every model from Tasks 3 and 4. Metrics are **pooled over out-of-fold predictions** rather than averaged over folds, because `family_group` folds are badly uneven (106 to 246 rows); a fold mean would silently weight a 106-row fold equally with a 246-row one. Both are reported.

The **optimism gap** — random-split Spearman minus grouped-split Spearman — is computed per model as the credit a model earns purely for recognising proteins and families it has already seen.

Command.

```
python -m src.evaluate --config config/config.yaml
```

Outputs. `results/metrics/model_comparison.csv`, `evaluation_summary.{md,json}`, and figures `pred_vs_actual.png`, `residual_distribution.png`, `strategy_comparison.png`, `model_comparison{,_v2}.png`, `baseline_vs_nn_fit.png`, `baseline_vs_dual_branch_fit.png`.

Key result. Validation strategy matters more than model choice.

Strategy	Ridge	Random forest	Gradient boosting
`random`	0.644	0.627	0.650
`protein_group`	0.637	0.625	0.656

Strategy	Ridge	Random forest	Gradient boosting
`family_group`	0.507	0.522	0.556

(stability Spearman)

The optimism gap reaches +0.137 Spearman for ridge on stability. A random split would have reported ridge at 0.644 — competitive with gradient boosting's 0.650 — when its true out-of-family performance is 0.507. Protein grouping alone is nearly free (gap ≤ 0.007); it is *family* grouping that exposes the inflation, because related proteins share embeddings.

Pearson consistently exceeds Spearman on stability, indicating the models capture the linear trend better than the exact ordering — and ordering is what a prioritization task actually consumes, which is why Spearman is the selection metric.

Task 6 — Ablation study

Objective. Quantify each feature block's contribution by removing it.

Implementation. `src/ablation.py` . Seven feature subsets (A–G), each a full re-fit of the selected model under `family_group` — 70 refits, no reuse of a model trained on different inputs.

Command.

```
python -m src.ablation --config config/config.yaml
```

Outputs. `results/metrics/ablation_summary.{csv,md}` , `results/figures/ablation_performance_drop.png` .

Key result (Spearman, Δ vs full model D).

Subset	Features	`stability_ddG`	`functional_score`
D full	22 tabular + 256 emb	0.556	0.747
E all tabular, no embeddings	22	0.546 *(–0.010)*	0.723 *(–0.025)*
F full minus deltas	16 + 256	0.536 *(–0.021)*	0.717 *(–0.030)*
G full minus structural	15 + 256	0.413 *(–0.143)*	0.370 *(–0.377)*
C tabular + embeddings, no structural/deltas	9 + 256	0.397 *(–0.159)*	0.327 *(–0.421)*
A tabular only, no embeddings	9	0.373 *(–0.183)*	0.271 *(–0.477)*
B embeddings only	256	**–0.205** *(–0.762)*	**–0.110** *(–0.857)*

Two conclusions, both load-bearing for the rest of the project:

- **The structural block dominates.** Removing it costs –0.143 and –0.377 Spearman. It is the single most valuable group of features.
- **All 256 embedding dimensions buy 0.010–0.025 Spearman, and alone they are worse than useless** — subset B produces *negative* rank correlation. This confirms the Task 1 reading: a per-protein-constant vector cannot rank mutations within a protein. The finding is reported as measured rather than softened.

Task 7 — Model interpretation & feature attribution

Objective. Identify which features drive predictions, aggregated across folds.

Implementation. `src/attribution.py`, two complementary strategies behind an `ImportanceStrategy` registry:

- **Permutation importance** on *held-out* rows, shuffled **before** the pipeline so importance is defined on the original named columns. The 256 embedding columns are shuffled as **one unit**, since shuffling correlated columns individually would understate a block whose members substitute for each other.
- **Impurity importance**, reported alongside with its biases stated: it is training-set based, and it is defined on post-PCA inputs (`embedding_pc{i}`), not on the original columns.

Both are aggregated over the 5 `family_group` folds with mean and standard deviation.

Command.

```
python -m src.attribution --config config/config.yaml
```

Outputs. `results/metrics/feature_importances.csv`, `feature_group_importances.csv`, `feature_attribution.md`, and figures `feature_importance.png`, `feature_group_importance.png`.

Key result (permutation, mean over 5 folds).

`stability_ddG` — distributed across three blocks:

Rank	Feature	Group	Importance	Share
1	`solvent_accessibility`	structural	0.1076 ± 0.0305	22.8%
2	`delta_hydrophobicity`	deltas	0.1068 ± 0.0334	22.6%
3	`mutant_hydrophobicity`	physicochemical	0.0923 ± 0.0213	19.5%

`functional_score` — concentrated in one feature:

Rank	Feature	Group	Importance	Share
1	`distance_to_active_site`	structural	0.3145	**49.2%**
2	`abs_delta_charge`	deltas	0.1738	27.2%
3	`abs_delta_hydrophobicity`	deltas	0.0668	10.4%

The embedding block's permutation importance is −0.0020 (stability) and +0.0004 (function) — statistically indistinguishable from zero. Two independent methods, ablation and permutation, converge on the same verdict about the embeddings. That agreement is the strongest interpretation result in the project.

Note the divergence between the two targets: function is driven overwhelmingly by *where* the mutation sits relative to the active site, stability by *what* the substitution changes physicochemically. The two targets are not interchangeable and should not share a single ranking signal uncritically.

Task 8 — Candidate prioritization & uncertainty quantification

Objective. Predict both targets for the 180 unlabeled candidates, quantify uncertainty, rank under a multi-objective criterion, and flag out-of-distribution candidates.

Implementation. `src/predict.py` .

1. **Ensemble inference** — the mean of the 5 `family_group` fold pipelines, with their **standard deviation as the uncertainty estimate**.
2. **Combined uncertainty** — the two per-target spreads added in quadrature.
3. **Viability gate (hard constraint)** — a candidate is viable only if predicted `ddG ≤ 0` **and** `functional_score ≥ 0.7` . The gate sorts **before** the soft score, so no weighting can promote a non-viable candidate above a viable one.
4. **Priority score (soft)** — rank-normalised objectives within the candidate pool, weights 0.5 / 0.5, minus 0.25 × normalised uncertainty.
5. **OOD flags** — candidates from PF08/PF09, absent from training entirely.

Every threshold above lives in `config/config.yaml` under `prioritization:` .

Command.

```
python -m src.predict --config config/config.yaml
```

Outputs. `results/mutation_rankings/top_10_candidates.{csv,md}` , `candidate_predictions.csv` (all 180, fully scored).

Key result. Verified: zero NaNs, and the ranking is deterministic — ties break on `[is_viable, priority_score, mutation_id]` .

Rank	Candidate	Mutation	Family	pred ddG	pred function	Viability
1	CAND0148	S490L	PF06	−0.013 ± 0.085	0.861 ± 0.027	pass
2	CAND0162	Y12F	PF03	−0.149 ± 0.038	0.800 ± 0.013	pass
3	CAND0167	N133S	PF03	−0.068 ± 0.125	0.854 ± 0.013	pass
4	CAND0015	E353V	PF03	−0.076 ± 0.089	0.750 ± 0.014	pass
5	CAND0126	Q360F	PF06	−0.019 ± 0.156	0.798 ± 0.012	pass
6	CAND0002	M496W	PF02	+0.234 ± 0.040	0.878 ± 0.013	fails_stability

Only 5 of 180 candidates clear the stability gate. This is the single most important operational finding of Task 8, and it is the reason `exclude_non_viable` is set to `false` : excluding non-viable candidates would return a Top 5, not a Top 10. Ranks 6–10 are reported *and explicitly labelled* *`fails_stability`* , so the shortlist stays the requested length while the gate remains visible. CAND0002 carries the **highest priority score in the entire pool (0.885)** yet ranks 6th, because the gate sorts first — the constraint overriding the score, visible in one row.

Task 9 — Scientific interpretation

Objective. Select 3–5 interesting mutations and discuss, for each, why the model considers it promising or unfavourable, the important input features, a structural/physicochemical interpretation, the prediction confidence, and any scientific concerns.

Implementation. `src/interpret.py` generates the per-candidate narrative from the scored artifact. The written analysis is in `README.md` § *Main results → Scientific interpretation* and in `report/final_report.md`.

Command.

```
python -m src.interpret --config config/config.yaml
```

Outputs. `results/mutation_rankings/candidate_interpretation.md`.

Key result. Five candidates were selected to span deliberately different scientific situations rather than simply the top five:

Candidate	Why it was selected
CAND0162 Y12F (rank 2)	The **only** shortlisted prediction whose stabilizing effect exceeds its own uncertainty (|ddG|/σ = **3.91**; the other four are 0.12–0.85). Near-isosteric, Δvolume −3.7 Å³, on an exposed coil (SASA 0.940) at low conservation (0.186) — a conservative substitution in a tolerant position.
CAND0148 S490L (rank 1)	Ranked **first**, yet its sign is unresolved — the effect is ~6× smaller than its spread. The physicochemistry argues *against* the model: Δhydrophobicity +4.6 and Δvolume +77.7 Å³ on an exposed helix face, where burying a large hydrophobe is entropically unfavourable. The entry most in need of measurement rather than trust.
CAND0126 Q360F (rank 5)	Largest stability uncertainty in the shortlist (±0.156, ~8× the effect). It shares protein PROT022 with rank 1, so **ranks 1 and 5 are not independent evidence**.
CAND0002 M496W (rank 6)	Highest priority score in the pool (0.885) but ranked 6th by the viability gate. Demonstrates the hard constraint overriding the soft score.
CAND0091 Y69W (rank 14)	Best-ranked **unseen-family** candidate (PF08), with function 0.870 ± **0.004** — the *tightest* uncertainty of all five, on the one mutation where the model has least right to be confident.

The fifth case carries the project's central negative result: **ensemble spread measures whether the five fold models agree, not whether they are jointly wrong.** On a novel family they agree precisely because they share a blind spot. Only the explicit `ood_flag` protects against this — never the uncertainty number.

Overriding scientific concern. The dataset is declared *fully synthetic* by its own README ("This dataset is fully synthetic and created only for technical assessment"), with both targets labelled synthetic field by field. The data's own fingerprints corroborate it: the embeddings are standard-normal noise (36 × 256, mean −0.004, sd 0.886) and amino-acid usage is uniform (3.8–6.0% across all 20 residues, against a real proteome's sharp skew). No biophysical conclusion above transfers to a real protein. What transfers is the methodology — the leakage control, the ablation design and the uncertainty audit.

Task 10 — Research extension

Objective. Given full sequences, AlphaFold structures, MSAs, MD trajectories, larger experimental datasets and additional GPUs, describe the next-generation architecture. *The brief states explicitly that this section need not be implemented.*

Implementation. None, by design. The written proposal is in `README.md` § *Research extension*.

Command. None — documentation only.

Key content. Nine steps, in the order they would actually be worked, each justified by a measurement from this project rather than offered as a menu:

1. **Residue-level ESM representations** with zero-shot log-odds features — attacks the per-protein-constant failure that ablation *and* permutation independently confirmed. Re-run the ablation before training anything larger.
2. **Foundation-model fine-tuning** — frozen trunk → LoRA → full, with family-grouped CV as the stopping criterion, because this project measured random-split inflation at +0.137.
3. **GNN over a residue contact graph, then SE(3)-equivariant** — the structural block dominates both attribution analyses; the dual-branch shape is already correct, so the embedding branch is swapped for a structural encoder.
4. **Multimodal fusion with per-task loss scaling from the start** — carrying forward the measured lesson that separating branches was necessary but *not sufficient* until the loss weighting was fixed.
5. **MSA-derived PSSM and Potts/DCA couplings**, carrying MSA depth as an explicit feature.
6. **MD-derived RMSF, SASA variance and local unfolding** — aimed at the weaker stability target (R^2 0.34 vs 0.63).
7. **Transfer learning from ProTherm/ProteinGym** with batch effects modelled, plus contrastive pre-training that separates same-protein mutations.
8. **Calibrated uncertainty (conformal) *before* active learning** — order matters: an acquisition function blind to novelty under-explores exactly the families that need it.
9. **Generative design last**, because a generator is only as good as its oracle, and this oracle cannot resolve 0.4 kcal/mol.

What would be kept unchanged. The grouped CV, the leakage assertions, the ablation harness and config-driven reproducibility. These are not scaffolding to discard once better models arrive — they are what would make a better model's improvement believable.

A note on scope, recorded deliberately: a message-passing network was considered for implementation and **declined on evidence**. An MPNN requires a graph whose edges encode real physics, and this dataset supplies no coordinates, no sequences, no contact map and no MSAs. `distance_to_active_site` is a distance to a single reference point, from which pairwise distances are not recoverable even in principle. Any graph would therefore have fabricated edges, and the ablation already predicts the benefit is approximately zero. Implementing one would have been the only model in the repository whose inputs were not real.

3. Verification

```
python -m pytest -q             # test suite
python -m ruff check .          # lint
python -m mypy src main.py      # static types
```

Current status, reported as measured on 2026-11-05:

Check	Result
`ruff check .`	clean
`mypy src main.py`	clean (22 source files)

Check	Result
`pytest -q`	**807 passed, 0 failed** in 81 s

Two API defects found and fixed

`tests/test_api.py` previously *skipped* because FastAPI was not installed. Installing the web dependencies made its 24 tests execute, and 6 of them failed, exposing two independent defects in `src/api.py`. Both are now fixed.

1. **`/predict` raised `KeyError: 'total_uncertainty'`.** `ModelService.score()` ended at `apply_viability_gate()`, but `to_prediction()` reads `COL_TOTAL_UNCERTAINTY` — a column added by `score_candidates()`, which `score()` never called.
   Fix. The quadrature sum was extracted into `combine_uncertainty()` in `src/predict.py` and is now called by both `score_candidates()` and the API. The split is deliberate: `total_uncertainty` is the one uncertainty quantity that is **absolute** — it depends on the row alone — whereas the normalised uncertainty, priority score and rank are batch-relative. Routing `/predict` through the full `score_candidates()` would have given a single mutation a meaningless batch-relative score instead.
2. **`/prioritize` raised `KeyError: 'mutation_id'`.** `scored.set_index(COL_MUTATION_ID)` consumed the column, which `to_prediction()` then read back off the row.
   Fix. `set_index(..., drop=False)`, so the column survives becoming the index.

Verification of the fix.

- Full suite: **807 passed, 0 failed**; `ruff` and `mypy` clean.
- The refactor is **behaviour-preserving**: re-running `python -m src.predict` reproduces `top_10_candidates.csv` and `candidate_predictions.csv` **byte-for-byte** (MD5 unchanged), so no scientific result in this report moved.
- Live end-to-end check: `/predict` returns HTTP 200 with `total_uncertainty = 0.04316`, equal to $\sqrt{(0.04084^2 + 0.01396^2)}$ — the quadrature is correct. `/prioritize` returns 200 and ranks correctly.
- Documented limitation, measured rather than assumed: a **single**-mutation `/prioritize` request returns a constant `priority_score` of 0.375 whatever the mutation (verified across substitutions spanning ddG +0.06 to +0.31 and function 0.58 to 0.89), because a one-row batch has nothing to rank against. `/predict` with `is_viable` is the correct call for a single mutation. This is now stated in `README.md` § *HTTP API*.

No pipeline stage or scientific result was affected at any point: `src/api.py` is an optional service layer that no task above depends on, and the batch path in `src/predict.py` always computed the column correctly.

4. Deliverables map

Deliverable	Location
Task-wise report *(this document)*	`report/task_wise_report.md`
Technical report	`report/final_report.md`, `report/final_report.pdf`
Repository guide	`README.md`
EDA notebook	`notebooks/exploratory_analysis.ipynb`

Deliverable	Location
Metrics	`results/metrics/`
Figures	`results/figures/`
Rankings	`results/mutation_rankings/`
Configuration	`config/config.yaml`